

# Thor Pipelines – Knowledge Transfer Document

## Overview

This document provides guidance on supporting Thor-related pipelines, including HipPlanning, Thor-Windows, Thor-Linux activities and mako4simulator. It covers common operational tasks, SonarQube report generation, production build support, and troubleshooting procedures.

## 1. HipPlanning Support

### 1.1 Add Components to JFrog Artifactory and Generate Production Build

#### Objective

Upload required components to JFrog Artifactory so that the Thor team can generate a production build based on the latest changes.

#### Procedure

Execute the following PowerShell command to upload the required component to Artifactory: Run below command in the powershell to add components

```
curl.exe -H "X-JFrog-Art-Api:AKCp5e2WkWPUvKRJtRNHfndA2r3ZN4rMUah6hZPXxSMN29aEviBLpB7T9nvWMR66bRAxX" -X PUT "https://artifactory.osep.stryker.com/artifactory/robotics-maven-dev/com/strykercorp/windows/x86_64/msvc15/tensorrt/11.10.1/tensorrt-11.10.1.pom" -T tensorrt-11.10.1.pom
```

```
curl.exe -H "X-JFrog-Art-Api:<API_KEY>" -X PUT "<ARTIFACTORY_URL>" -T tensorrt-11.10.1.pom
```

Upload the required artifacts under the following repository path:

[https://artifactory.osep.stryker.com/ui/native/robotics-maven-dev/com/strykercorp/windows/x86\\_64/msvc15/](https://artifactory.osep.stryker.com/ui/native/robotics-maven-dev/com/strykercorp/windows/x86_64/msvc15/)

🎬 Verify that all required files are successfully uploaded.

🎬 Once the upload is completed, coordinate with the Thor team to generate the production build using the newly added components.

reference link:

[Pipeline #2608365744 · strykercorp / IMT / HipPlanning · GitLab](#)

### 1.2 Generate SonarQube Report

#### Objective

Generate a SonarQube report for newly implemented code changes.

## Procedure

1. Use the **HPL\_DEV\_2\_1\_Sonar** branch.
2. Locate the **build.ps1** script within the repository.
3. If Sonar analysis is required for a different branch:
  - Update the branch name in the build.ps1 file.
  - Save the changes.
4. Execute the pipeline.
5. Open the pipeline:

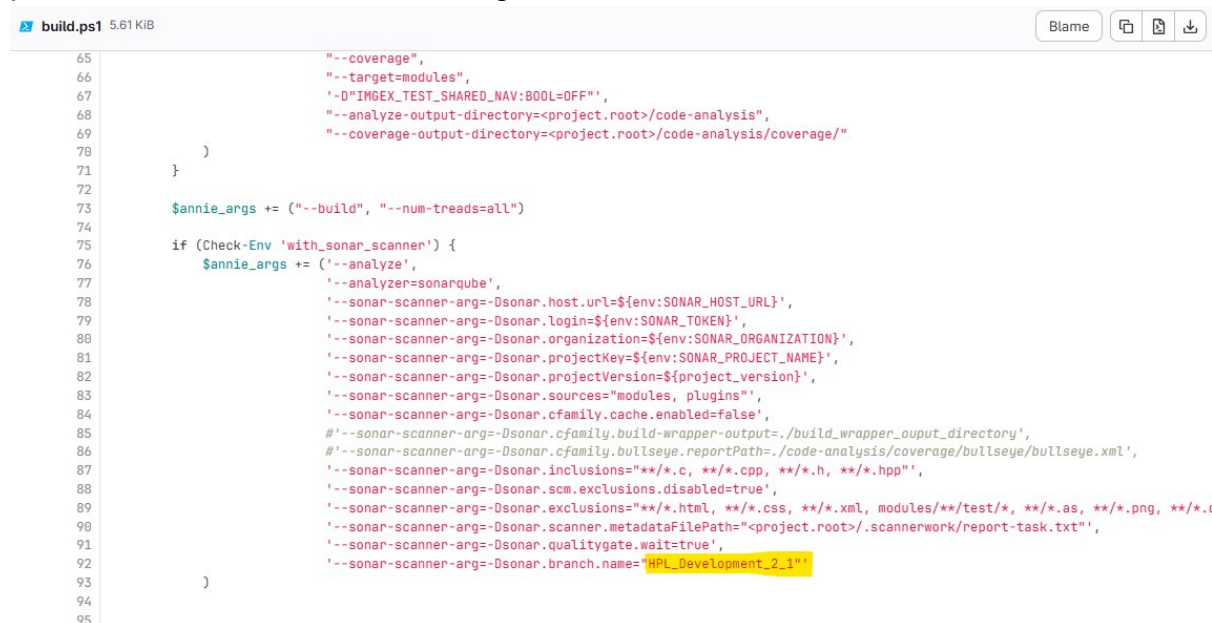
## hipplanning-docker-build

6. After successful completion, locate the generated SonarQube report URL.
7. Download the report and share it with the Thor team for review.

## Notes

- Ensure the correct branch is configured before triggering the pipeline.
- Refer to the project screenshots for branch configuration examples.

please check below reference image:



```
65         "--coverage",
66         "--target=modules",
67         "-D\"IMGEX_TEST_SHARED_NAV:BOOL=OFF\"",
68         "--analyze-output-directory=<project.root>/code-analysis",
69         "--coverage-output-directory=<project.root>/code-analysis/coverage/"
70     )
71 }
72
73 $annie_args += ("--build", "--num-treads=all")
74
75 if (Check-Env 'with_sonar_scanner') {
76     $annie_args += ("--analyze",
77         '--analyzer=sonarqube',
78         '--sonar-scanner-arg=Dsonar.host.url=$(env:SONAR_HOST_URL)',
79         '--sonar-scanner-arg=Dsonar.login=$(env:SONAR_TOKEN)',
80         '--sonar-scanner-arg=Dsonar.organization=$(env:SONAR_ORGANIZATION)',
81         '--sonar-scanner-arg=Dsonar.projectKey=$(env:SONAR_PROJECT_NAME)',
82         '--sonar-scanner-arg=Dsonar.projectVersion=$(project_version)',
83         '--sonar-scanner-arg=Dsonar.sources="modules, plugins"',
84         '--sonar-scanner-arg=Dsonar.cfamily.cache.enabled=false',
85         #'--sonar-scanner-arg=Dsonar.cfamily.build-wrapper-output=./build_wrapper_output_directory',
86         #'--sonar-scanner-arg=Dsonar.cfamily.bullseye.reportPath=./code-analysis/coverage/bullseye/bullseye.xml',
87         '--sonar-scanner-arg=Dsonar.inclusions="**/*.c, **/*.cpp, **/*.h, **/*.hpp"',
88         '--sonar-scanner-arg=Dsonar.scm.exclusions.disabled=true',
89         '--sonar-scanner-arg=Dsonar.exclusions="**/*.html, **/*.css, **/*.xml, modules/**/test/*, **/*.as, **/*.png, **/*.c"',
90         '--sonar-scanner-arg=Dsonar.scanner.metadataFilePath="<project.root>/scannerwork/report-task.txt"',
91         '--sonar-scanner-arg=Dsonar.qualitygate.wait=true',
92         '--sonar-scanner-arg=Dsonar.branch.name="HPL_Development_2_1"'
93     )
94 }
95
```

Below is the Reference pipeline URL:

[hipplanning-docker-build \(#13444352032\) · Jobs · strykercorp / IMT / HipPlanning · GitLab](#)

```
3832 WARN: * plugins/com.stryker.thor_planning/src/popup_slider_model.cpp
3833 WARN: * plugins/com.stryker.thor_planning/src/export_planning_handler.cpp
3834 WARN: * modules/implant_db/sqlite_base_api/include/implant_db/sqlite_base_api/aux_api/implant_family.h
3835 WARN: * plugins/com.stryker.thor_planning/src/proximity_detection_dialog.h
3836 WARN: This may lead to missing/broken features in SonarCloud
3837 INFO: Analysis report generated in 1851ms, dir size=15 MB
3838 INFO: Analysis report compressed in 3441ms, zip size=4 MB
3839 INFO: Analysis report uploaded in 1236ms
3840 INFO: ----- Upload SCA dependency files
3841 INFO: Sensor cache published successfully
3842 INFO: ----- Check Quality Gate status
3843 INFO: Waiting for the analysis report to be processed (max 300s)
3844 INFO: QUALITY GATE STATUS: PASSED - View details on https://sonarcloud.io/dashboard?id=imt%3Ahipplanning&branch=HPI\_Development\_2\_1
3845 INFO: Analysis total time: 23:31.706 s
3846 INFO: -----
3847 INFO: EXECUTION SUCCESS
3848 INFO: -----
3849 INFO: Total time: 23:36.555s
3850 INFO: Final Memory: 21M/96M
3851 INFO: -----
3852 Duration config/build/test/package: 02:16:53.1140156
3853 $ if ( ${env:with_package} -ne "True" ) { # collapsed multi-line command
3854 Package generation set to False. Exiting ...
3855 Cleaning up project directory and file based variables
3856 Job succeeded
```

To download the Sonar report, click on the generated URL as shown in the image above.

Once the report is downloaded, please share it with the Thor team.

## 2. Thor-Windows Support

### 2.1 Black Duck Scan Support

The Thor team may request the **install-root** folder from the latest build to perform Black Duck security scans.

#### Procedure

1. Obtain the install-root folder from the latest successful build.
2. Verify the folder contents.
3. Share the package with the Thor team.

Reference Pipeline:

[knee1-0\\_build \(#14477013279\) · Jobs · strykercorp / IMT / Navigation / Thor-Windows · GitLab](#)

### 2.2 Common Docker Build Issue

#### Error

docker: Error response from daemon:

The container operating system does not match the host operating system.

#### Root Cause

The Docker image OS version is incompatible with the Windows host operating system.

## Resolution

1. Create a new Docker image using an appropriate Windows base image.
2. Install the required MSVC Build Tools and dependencies.
3. Build and validate the image locally.
4. Update the build image reference in the GitLab CI configuration.
5. Re-run the pipeline.

```
48 WARNING! Your password will be stored unencrypted in C:\Users\oseprunner\shell_home\config.json.  
49 Configure a credential helper to remove this warning. See  
50 https://docs.docker.com/engine/reference/commandline/login/#credentials-store  
51 Login Succeeded  
52 $ docker run --rm --name=${container_name} -v ${CI_PROJECT_DIR}:${workdir} -w "${workdir}/modules/perforce" -e PYTHONDONTWRITEBYTECODE=1 --entrypoint=python -t ${perforce_image} checkout.py --p4port ${P4PORT} --p4user ${P4USER} --p4passwd ${P4PASSWD} --stream ${P4_STREAM} --dest c:/src/workspace --name ${client_name} --output /src/changed_code.txt  
53 docker: Error response from daemon: hcs::CreateComputeSystem 856c11127cfab04aacf32e2f408be736cec34ade8e69a6a8fa63ad28b59178d3: The container operating system does not match the host operating system.  
54 Running after_script  
55 Running after script...  
56 $ cd $CI_PROJECT_DIR  
57 $ docker run --rm -v ${CI_PROJECT_DIR}:${workdir} -w "${workdir}/modules/perforce" -e PYTHONDONTWRITEBYTECODE=1 --entrypoint=python -t ${perforce_image} remove_client.py --p4port ${P4PORT} --p4user ${P4USER} --p4passwd ${P4PASSWD} --p4client ${client_name}  
58 docker: Error response from daemon: hcs::CreateComputeSystem 83b3ef4bc4c891c817866407cec31716a6b2c5d84f167d9ae08f568cc8937105: The container operating system does not match the host operating system.  
59 WARNING: after_script failed, but job will continue unaffected: exit status 125  
60 Cleaning up project directory and file based variables  
61 ERROR: Job failed: exit status 125
```

Reference Dockerfile:

[Dockerfile.windows-python-3.11.qt5.12.2.vc16.qtc-14 · upload\\_zip\\_folder\\_to\\_artifact-registry · strykercorp / IMT / Navigation / Thor-Windows · GitLab](#)

## Additional Reference

JFrog Docker Images Repository:

<https://artifactory.osep.stryker.com/ui/repos/tree/General/docker-dev/qbs-build-tools/>

Use the existing Windows images as a reference to troubleshoot and resolve the above Thor-Windows error.

## 3. Thor-Linux Support

### Overview

Thor-Linux primarily uses scheduled release pipelines.

Reference Pipeline:

[strykercorp / IMT / Navigation / Thor-Linux · GitLab](#)

### Release Pipeline

- thor\_cknee2.0\_release

## **Support Expectations**

1. Monitor scheduled pipeline executions.
2. Verify release status and deployment progress.
3. Address any build or release failures.
4. Coordinate with the Thor team when intervention is required.

## **Important Note**

Most release activities are performed during late evening hours, typically around 9:00 PM. Ensure availability during release windows to provide timely support and issue resolution.

## **4. Mako4Simulator Support**

### **Overview**

The Mako4Simulator pipeline was primarily used for generating SonarQube reports during the implementation phase of the project. Currently, there have been no recent changes or enhancement requests related to this pipeline.

Reference Pipeline:

[analyze\\_build \(#12763768596\) · Jobs · strykercorp / Digital Robotics and Enabling Technologies / Mako4 / Mako4Simulator · GitLab](#)

### **Purpose**

**The pipeline was utilized to:**

- Execute code quality analysis using SonarQube.
- Generate SonarQube reports for newly implemented features and code changes.
- Review code quality metrics, vulnerabilities, code smells, and coverage reports.

**Procedure for SonarQube Report Generation**

1. Navigate to the analyze\_build pipeline.
2. Trigger the pipeline for the required branch.
3. Wait for the SonarQube analysis stage to complete successfully.
4. Access the generated SonarQube report URL from the pipeline output.
5. Download or review the report and share it with the respective team if requested.

**Current Status**

- The pipeline remains available for SonarQube report generation.
- No recent modifications or maintenance activities have been performed.
- No outstanding issues or active development efforts are currently associated with this pipeline.